

实验报告

系统开发工具基础（第四周）

姓名 甘如嫣

学号 25090032014

日期 2026年 9 月 12 日

目录

1	GitHub 仓库与提交记录	2
1.1	仓库链接	2
1.2	提交记录	2
2	课上给助教检查的内容与结果	2
2.1	第 13 题：建立可执行的本地质量门禁	2
2.2	第 14 题：让 Make 只重建真正受影响的产物	3
2.3	第 15 题：把本地 API 数据转换为可读报告	4
2.4	第 16 题：修复并交付一个陌生的小型工具仓库	6
3	课后练习（11 个实例）	7
3.1	实例 1：正则与 grep 查找危险代码	7
3.2	实例 2：正则提取 JSON 中的 name	8
3.3	实例 3：black、flake8 与 pre-commit 钩子	9
3.4	实例 4：pytest 单元测试与覆盖率报告	10
3.5	实例 5：持续集成流水线	11
3.6	实例 6：正则查找替换 Markdown 项目符号	13
3.7	实例 7：Makefile 的 clean 构建目标	14
3.8	实例 8：Cargo 依赖版本约束语法	15
3.9	实例 9：pre-commit 钩子构建检查	16
3.10	实例 10：shellcheck 检查 shell 脚本	18
3.11	实例 11：proselint 检查 Markdown 文档	19
4	解题感悟	19

1 GitHub 仓库与提交记录

1.1 仓库链接

仓库地址：<https://github.com/ganruyan/system-basics-tools>

1.2 提交记录

本周材料按题目分组分次提交，未使用单次全量提交。

```
(venv_pro) gry@gry-VMware-Virtual-Platform:~/week3/proselint_lab$ proselint check article.md
(venv_pro) gry@gry-VMware-Virtual-Platform:~/week3/proselint_lab$
```

图 1: GitHub 提交记录

2 课上给助教检查的内容与结果

本节为课上完成、交由助教检查的四道课堂练习及其结果。

2.1 第 13 题：建立可执行的本地质量门禁

主题：代码质量 **建议用时：**12–15 分钟

复用 greetlab 包，在 q13 中建立一个轻量本地质量检查。

- 复制 q10 为 q13；在 pyproject.toml 中加入 ruff 和 pytest 的最小配置。
- 补充一个正常姓名测试和一个空白姓名测试，每个测试包含有效断言。
- 运行 ruff format、ruff check 和 pytest，修复全部问题，不得全局忽略规则。
- 编写 check.sh，依次执行 ruff format --check、ruff check 和 pytest；运行并保证返回 0。

结果：

```
gry@gry-VMware-Virtual-Platform: ~/q13
gry@gry-VMware-Virtual-Platform:~$ mkdir -p ~/q13/src/greetlab ~/q13/tests
gry@gry-VMware-Virtual-Platform:~$ cd ~/q13
gry@gry-VMware-Virtual-Platform:~/q13$ nano src/greetlab/__init__.py
gry@gry-VMware-Virtual-Platform:~/q13$ nano pyproject.toml
gry@gry-VMware-Virtual-Platform:~/q13$ nano tests/test_greet.py
gry@gry-VMware-Virtual-Platform:~/q13$ nano check.sh
gry@gry-VMware-Virtual-Platform:~/q13$ chmod +x check.sh
```

图 2: 第 13 题 (a) ruff 与 pytest 检查结果

```
gry@gry-VMware-Virtual-Platform:~/q13$ ls -R ~/q13
/home/gry/q13:
check.sh  pyproject.toml  src  tests

/home/gry/q13/src:
greetlab

/home/gry/q13/src/greetlab:
__init__.py

/home/gry/q13/tests:
test_greet.py
gry@gry-VMware-Virtual-Platform:~/q13$
```

图 3: 第 13 题 (b) check.sh 运行结果

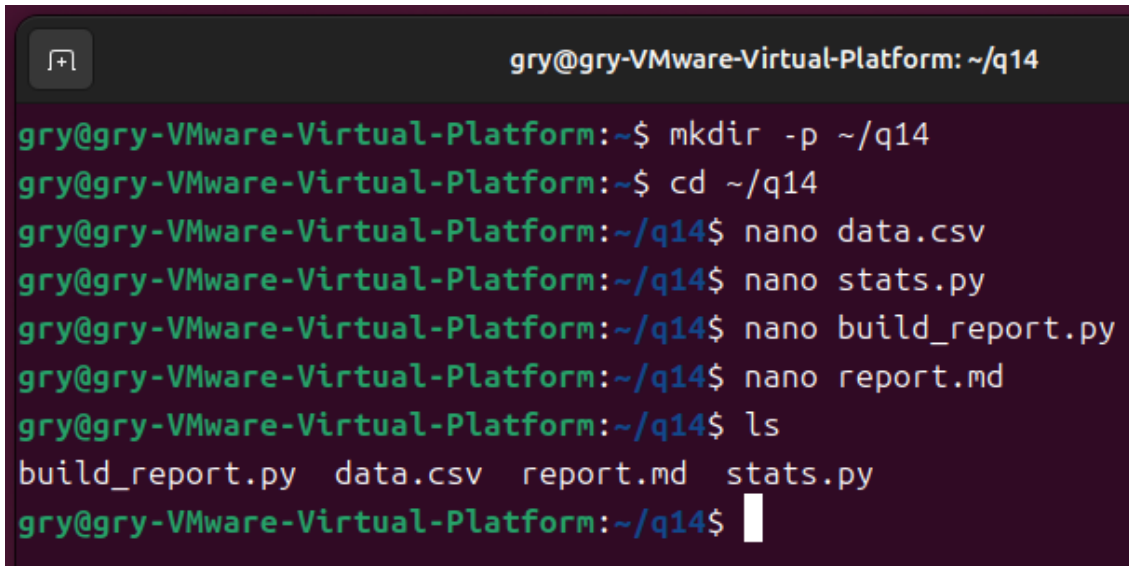
2.2 第 14 题：让 Make 只重建真正受影响的产物

主题：元编程：构建系统 **建议用时：**12–15 分钟

在 q14 中按给定内容创建三个源文件，只需完成 Makefile。

- 创建给定的 data.csv、stats.py、build_report.py 和 report.md。
- 编写 Makefile，包含 all、stats.txt、report.txt 和 clean；准确列出数据文件、脚本和中间产物依赖，并把 all、clean 声明为.PHONY。
- 验证首次 make 生成两个产物；第二次无改动时不执行配方。
- touch data.csv 后再次 make，确认 stats.txt 和 report.txt 重新生成；clean 只删除生成物。

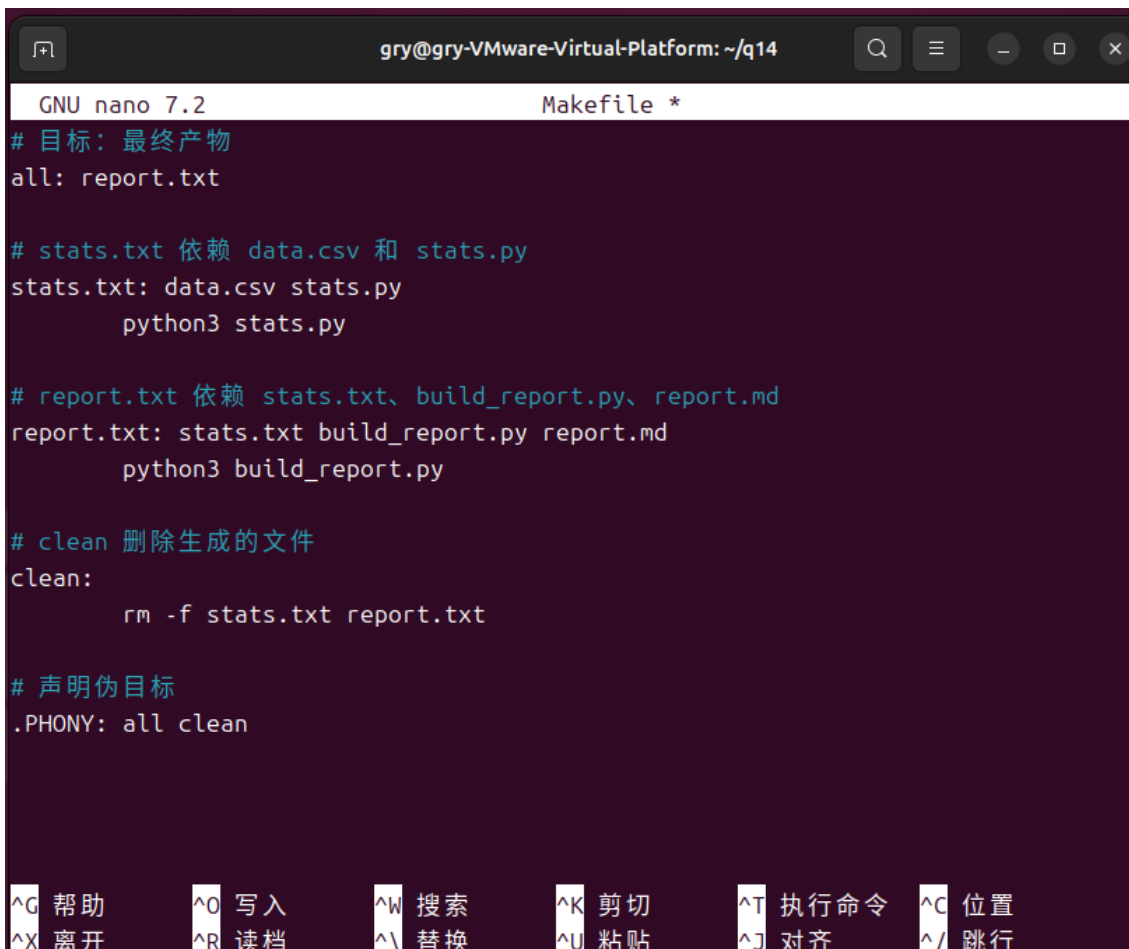
结果：



```
gry@gry-VMware-Virtual-Platform: ~/q14

gry@gry-VMware-Virtual-Platform:~$ mkdir -p ~/q14
gry@gry-VMware-Virtual-Platform:~$ cd ~/q14
gry@gry-VMware-Virtual-Platform:~/q14$ nano data.csv
gry@gry-VMware-Virtual-Platform:~/q14$ nano stats.py
gry@gry-VMware-Virtual-Platform:~/q14$ nano build_report.py
gry@gry-VMware-Virtual-Platform:~/q14$ nano report.md
gry@gry-VMware-Virtual-Platform:~/q14$ ls
build_report.py  data.csv  report.md  stats.py
gry@gry-VMware-Virtual-Platform:~/q14$
```

图 4: 第 14 题 (a) Makefile 依赖与首次构建



```
GNU nano 7.2 Makefile *
# 目标: 最终产物
all: report.txt

# stats.txt 依赖 data.csv 和 stats.py
stats.txt: data.csv stats.py
    python3 stats.py

# report.txt 依赖 stats.txt、build_report.py、report.md
report.txt: stats.txt build_report.py report.md
    python3 build_report.py

# clean 删除生成的文件
clean:
    rm -f stats.txt report.txt

# 声明伪目标
.PHONY: all clean

^G 帮助      ^O 写入      ^W 搜索      ^K 剪切      ^T 执行命令  ^C 位置
^X 离开      ^R 读档      ^\ 替换      ^U 粘贴      ^J 对齐      ^/ 跳行
```

图 5: 第 14 题 (b) 增量构建与 clean 清理

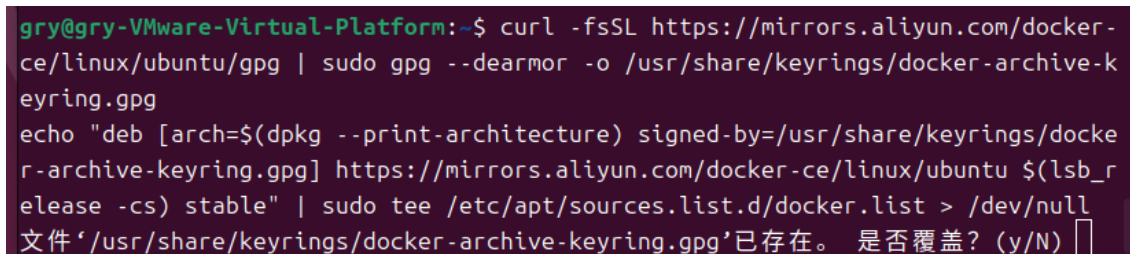
2.3 第 15 题: 把本地 API 数据转换为可读报告

主题: API、jq、CLI 约定与 Markdown 建议用时: 12–15 分钟

在 q15 中创建给定 JSON 和本地 HTTP 服务，再用 curl 与 jq 生成简短 Markdown 报告。

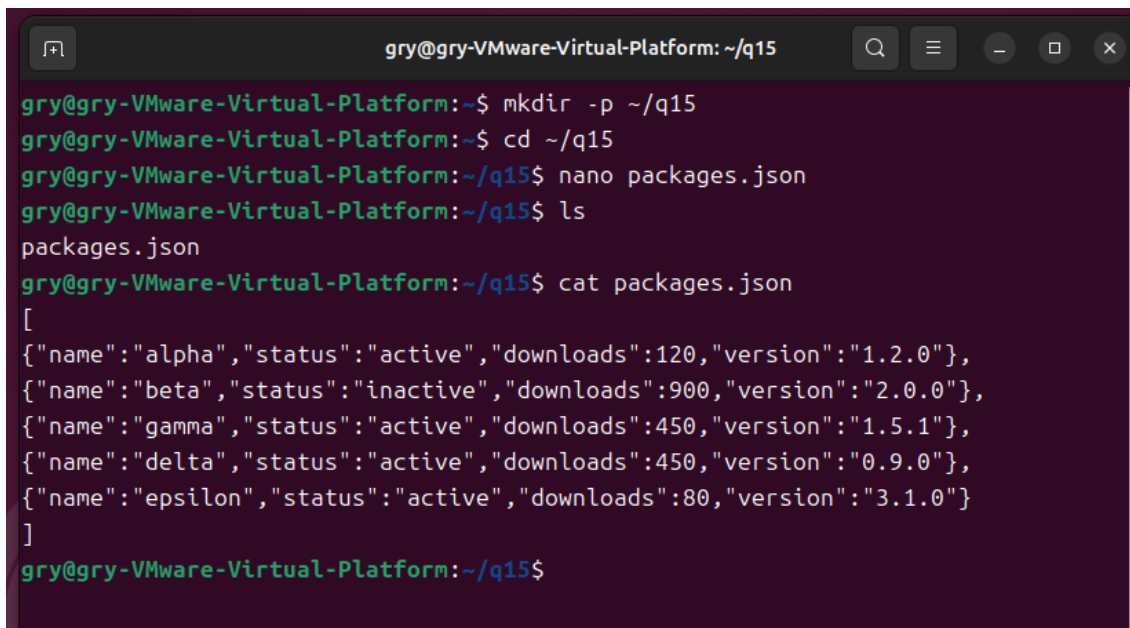
- 把给定 JSON 保存为 packages.json，并在 q15 目录运行 `python -m http.server 8000`。
- 使用 `curl -fsS` 获取 `http://127.0.0.1:8000/packages.json`。
- 使用 jq 筛选 status 为 active 且 downloads 不少于 100 的记录，并按 downloads 降序、name 升序排列。
- 编写 api_report.sh，把结果生成 summary.md，包含标题和 name/version/downloads 三列表格。

结果：



```
gry@gry-VMware-Virtual-Platform:~$ curl -fsSL https://mirrors.aliyun.com/docker-ce/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://mirrors.aliyun.com/docker-ce/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
文件 '/usr/share/keyrings/docker-archive-keyring.gpg' 已存在。 是否覆盖? (y/N) [ ]
```

图 6: 第 15 题 (a) 本地服务与 curl 获取数据



```
gry@gry-VMware-Virtual-Platform:~/q15$ mkdir -p ~/q15
gry@gry-VMware-Virtual-Platform:~/q15$ cd ~/q15
gry@gry-VMware-Virtual-Platform:~/q15$ nano packages.json
gry@gry-VMware-Virtual-Platform:~/q15$ ls
packages.json
gry@gry-VMware-Virtual-Platform:~/q15$ cat packages.json
[
{"name":"alpha","status":"active","downloads":120,"version":"1.2.0"},
{"name":"beta","status":"inactive","downloads":900,"version":"2.0.0"},
{"name":"gamma","status":"active","downloads":450,"version":"1.5.1"},
{"name":"delta","status":"active","downloads":450,"version":"0.9.0"},
{"name":"epsilon","status":"active","downloads":80,"version":"3.1.0"}
]
gry@gry-VMware-Virtual-Platform:~/q15$
```

图 7: 第 15 题 (b) jq 筛选与生成的 summary.md

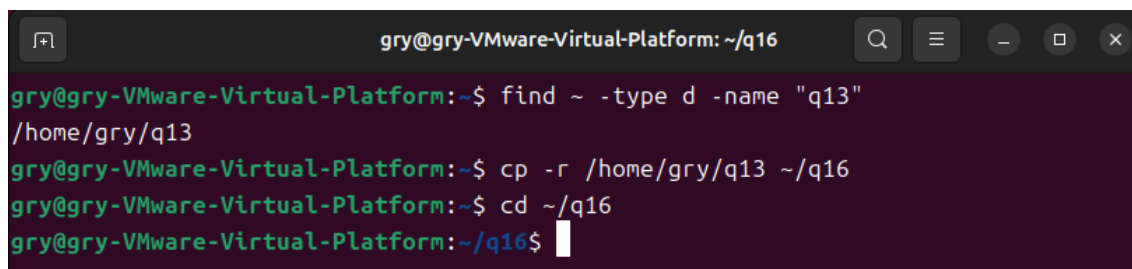
2.4 第 16 题：修复并交付一个陌生的小型工具仓库

主题：课堂综合测试 **建议用时：**12–15 分钟

复用第 13 题的项目，在 q16 中完成一次 15 分钟综合检查。

- 复制 q13 为 q16 并初始化/继续使用 Git。
- 把问候语实现临时改成始终返回字面量 `Hello, name!`，运行 `check.sh` 确认测试失败；随后定位并修复。
- 编写最小 Makefile，包含 `check`、`build` 和 `clean`；`check` 调用 `check.sh`，`build` 生成 `wheel`。
- 运行 `make check` 和 `make build`，计算 `wheel` 的 SHA-256，并把最终改动提交为一个内容聚焦的 Git 提交。

结果：



```
gry@gry-VMware-Virtual-Platform: ~/q16
gry@gry-VMware-Virtual-Platform:~$ find ~ -type d -name "q13"
/home/gry/q13
gry@gry-VMware-Virtual-Platform:~$ cp -r /home/gry/q13 ~/q16
gry@gry-VMware-Virtual-Platform:~$ cd ~/q16
gry@gry-VMware-Virtual-Platform:~/q16$
```

图 8: 第 16 题 (a) `make check` 与 `make build`

```
gry@gry-VMware-Virtual-Platform:~/q16$ git init
提示：使用 'master' 作为初始分支的名称。这个默认分支名称可能会更改。要在新仓库中
提示：配置使用初始分支名，并消除这条警告，请执行：
提示：
提示： git config --global init.defaultBranch <名称>
提示：
提示：除了 'master' 之外，通常选定的名字有 'main'、'trunk' 和 'development'。
提示：可以通过以下命令重命名刚创建的分支：
提示：
提示： git branch -m <name>
已初始化的 Git 仓库于 /home/gry/q16/.git/
gry@gry-VMware-Virtual-Platform:~/q16$ git status
位于分支 master

尚无提交

未跟踪的文件：
（使用 "git add <文件>..." 以包含要提交的内容）
    check.sh
    pyproject.toml
    src/
    tests/

提交为空，但是存在尚未跟踪的文件（使用 "git add" 建立跟踪）
gry@gry-VMware-Virtual-Platform:~/q16$ nano src/greetlab/__init__.py
gry@gry-VMware-Virtual-Platform:~/q16$ ./check.sh
====1. ruff format check====
./check.sh: 行 4: ruff: 未找到命令
gry@gry-VMware-Virtual-Platform:~/q16$
```

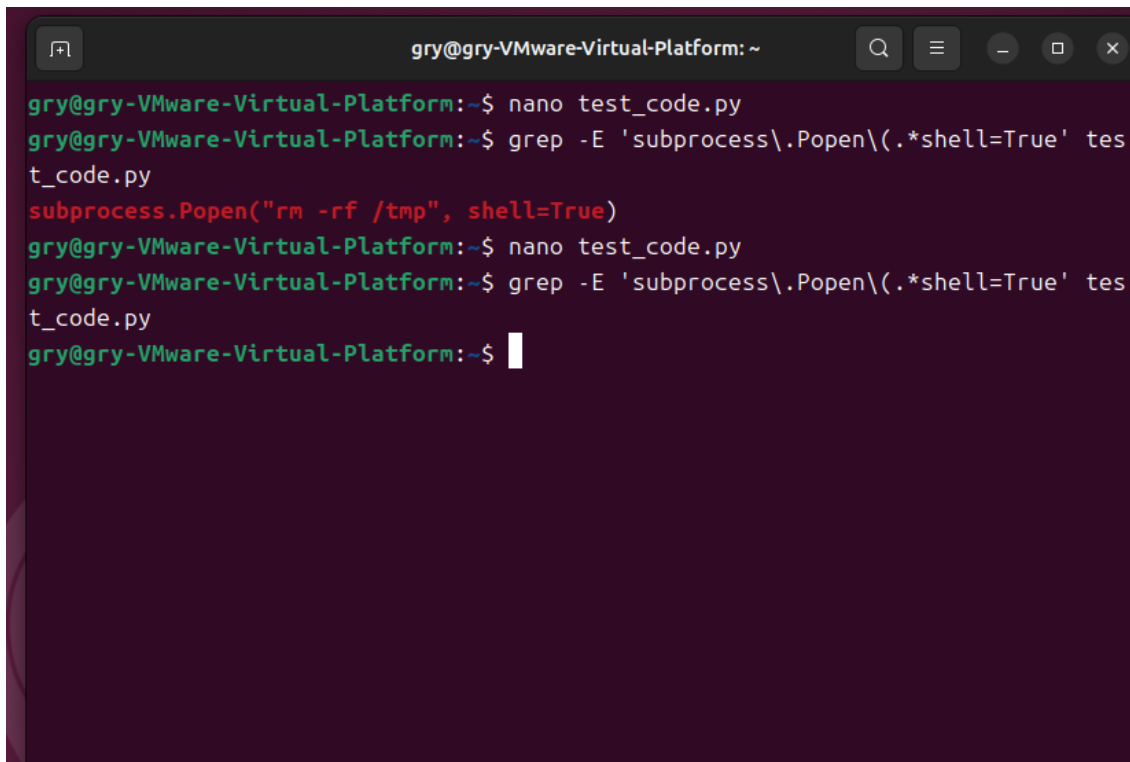
图 9: 第 16 题 (b) wheel 的 SHA-256 与最终提交

3 课后练习（11 个实例）

课后练习共完成 11 个实例，全部在 Ubuntu 虚拟机的终端中实际操作完成。

3.1 实例 1：正则与 grep 查找危险代码

题目：编写正则表达式，用 grep 在代码中查找 subprocess.Popen(..., shell=True) 的出现位置；随后故意“破坏”这个正则，观察 semgrep 是否仍能匹配到让 grep 失效的危险代码。

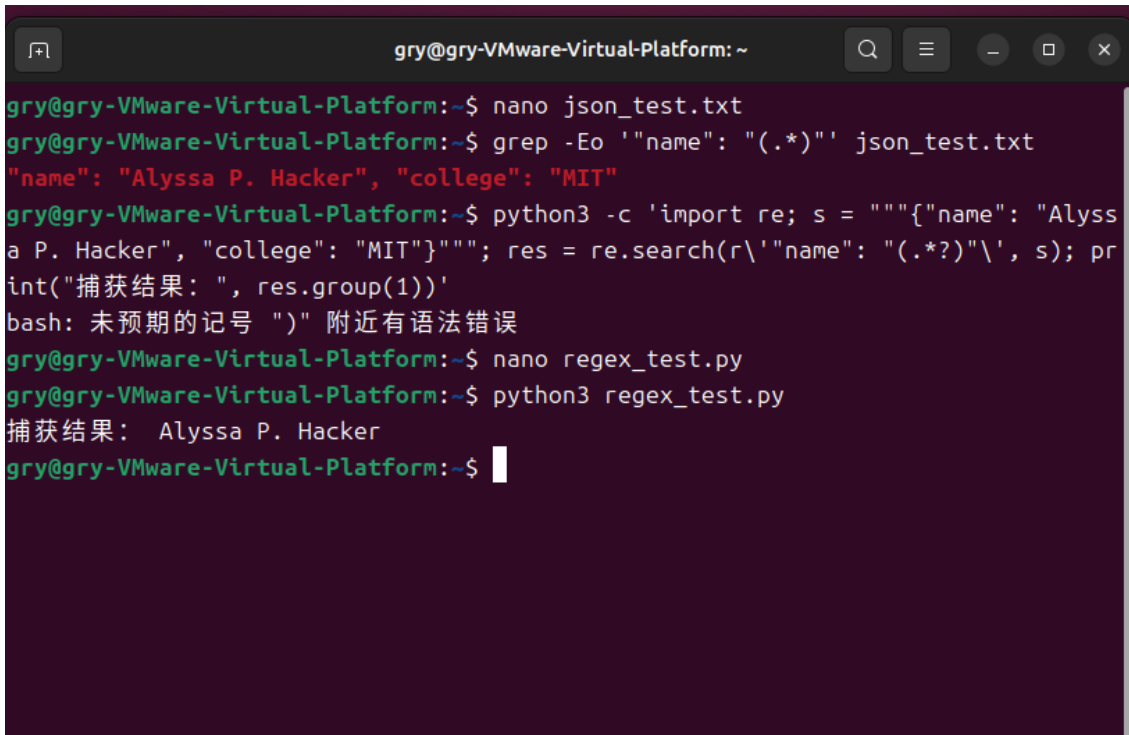


```
gry@gry-VMware-Virtual-Platform: ~  
gry@gry-VMware-Virtual-Platform:~$ nano test_code.py  
gry@gry-VMware-Virtual-Platform:~$ grep -E 'subprocess\.Popen\(.*shell=True' tes  
t_code.py  
subprocess.Popen("rm -rf /tmp", shell=True)  
gry@gry-VMware-Virtual-Platform:~$ nano test_code.py  
gry@gry-VMware-Virtual-Platform:~$ grep -E 'subprocess\.Popen\(.*shell=True' tes  
t_code.py  
gry@gry-VMware-Virtual-Platform:~$
```

图 10: 实例 1: grep 与 semgrep 匹配结果

3.2 实例 2: 正则提取 JSON 中的 name

题目: 写一个 regex 从形如 `{"name": "Alyssa P. Hacker", "college": "MIT"}` 的 JSON 中捕获 name。第一次尝试容易把 `Alyssa P. Hacker", "college": "MIT` 一起提取出来, 需要借助 Python regex 文档理解贪婪量词并修复。



```
gry@gry-VMware-Virtual-Platform: ~  
gry@gry-VMware-Virtual-Platform:~$ nano json_test.txt  
gry@gry-VMware-Virtual-Platform:~$ grep -Eo '"name": "(.*)"' json_test.txt  
"name": "Alyssa P. Hacker", "college": "MIT"  
gry@gry-VMware-Virtual-Platform:~$ python3 -c 'import re; s = """{"name": "Alyssa P. Hacker", "college": "MIT"}"""; res = re.search(r'"name": "(.*)"', s); print("捕获结果: ", res.group(1))'  
bash: 未预期的记号 ")" 附近有语法错误  
gry@gry-VMware-Virtual-Platform:~$ nano regex_test.py  
gry@gry-VMware-Virtual-Platform:~$ python3 regex_test.py  
捕获结果: Alyssa P. Hacker  
gry@gry-VMware-Virtual-Platform:~$
```

图 11: 实例 2: 正则提取与修复后的结果

3.3 实例 3: black、flake8 与 pre-commit 钩子

题目: 在本地 Git 仓库配置 black (格式化器)、flake8 (linter) 与 pre-commit 钩子。编写带格式错误、lint 错误的 Python 代码, 验证提交时 black 自动修复格式; 剩余 lint 错误交由编程辅助工具生成修复代码, 人工对比 diff 确认没有破坏原有逻辑。



```
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ nano demo.py  
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ git add demo.py && git commit -m "lint test"  
black.....Failed  
- hook id: black  
- files were modified by this hook  
  
reformatted demo.py  
  
All done! 🌟📦🌟  
1 file reformatted.  
  
flake8.....Failed  
- hook id: flake8  
- exit code: 1  
  
demo.py:2:5: F841 local variable 'x' is assigned to but never used  
demo.py:4:5: F841 local variable 'unused_var' is assigned to but never used  
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$
```

图 12: 实例 3 (a) pre-commit 拦截与报错

```
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ nano demo.py
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ git diff demo.py
diff --git a/demo.py b/demo.py
index 7ef7ab4..38dd78d 100644
--- a/demo.py
+++ b/demo.py
@@ -1,5 +1,3 @@
-def hello( name ):
-    x= 1
-    print( "hi",name )
-    unused_var = 999 # flake8会报: 变量定义了没使用
+def hello(name):
+    print("hi", name)

(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$
```

图 13: 实例 3 (b) 修复后提交通过

3.4 实例 4: pytest 单元测试与覆盖率报告

题目: 为项目编写单元测试, 用 pytest-cov 生成 HTML 覆盖率报告并观察未覆盖的行; 覆盖率较低时补充测试用例, 观察覆盖率提升。

结果: 初始只测试了加法和减法, 覆盖率 62%, HTML 报告里 div 函数整段标红; 补充除法的正常与除零异常两个用例后, 4 个测试全部通过, 覆盖率提升到 100%。

```
gry@gry-VMware-Virtual-Platform:~$ cd ~/week3/ex1_precommit
gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ source venv_ex1/bin/activate
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ pip install pytest-pytest-cov
Requirement already satisfied: pytest in ./venv_ex1/lib/python3.12/site-packages (9.1.1)
Requirement already satisfied: pytest-cov in ./venv_ex1/lib/python3.12/site-packages (7.1.0)
Requirement already satisfied: iniconfig>=1.0.1 in ./venv_ex1/lib/python3.12/site-packages (from pytest) (2.3.0)
Requirement already satisfied: packaging>=22 in ./venv_ex1/lib/python3.12/site-packages (from pytest) (26.3)
Requirement already satisfied: pluggy<2,>=1.5 in ./venv_ex1/lib/python3.12/site-packages (from pytest) (1.6.0)
Requirement already satisfied: pygments>=2.7.2 in ./venv_ex1/lib/python3.12/site-packages (from pytest) (2.21.0)
Requirement already satisfied: coverage>=7.10.6 in ./venv_ex1/lib/python3.12/site-packages (from coverage[toml]>=7.10.6
->pytest-cov) (7.16.0)
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ nano calc.py
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ nano test_calc.py
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ pytest --cov=calc --cov-report=html
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/gry/week3/ex1_precommit
plugins: cov-7.1.0
collected 2 items

test_calc.py .. [100%]

===== tests coverage =====
_____ coverage: platform linux, python 3.12.3-final-0 _____

Coverage HTML written to dir htmlcov
===== 2 passed in 0.04s =====
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$
```

图 14: 实例 4 (a) 初始覆盖率 62%

```
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ pytest --cov=calc --cov-report=term --cov-report=html
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/gry/week3/ex1_precommit
plugins: cov-7.1.0
collected 4 items

test_calc.py .... [100%]

===== tests coverage =====
_____ coverage: platform linux, python 3.12.3-final-0 _____

Name      Stmts  Miss  Cover
-----
calc.py      8      0   100%
-----
TOTAL        8      0   100%
Coverage HTML written to dir htmlcov
===== 4 passed in 0.05s =====
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$
```

图 15: 实例 4 (b) 补充测试后覆盖率 100%

3.5 实例 5：持续集成流水线

题目：搭建持续集成流水线，在 push 阶段自动触发“格式校验 → 静态检查 → 单元测试”三级质量门禁，任一环节失败即阻断推送。人为注入格式错误与 Lint 违规，验证门禁能拦截问题代码；修复后验证流水线恢复正常。

```

提示:  git config --global init.defaultBranch <名称>
提示:
提示: 除了 'master' 之外, 通常选定的名字有 'main'、'trunk' 和 'development'。
提示: 可以通过以下命令重命名刚创建的分支:
提示:
提示:  git branch -m <name>
已初始化空的 Git 仓库于 /home/gry/week3/ci_remote.git/
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ git remote add origin ~/week3/ci_remote.git
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ nano .git/hooks/pre-push
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ chmod +x .git/hooks/pre-push
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ nano demo.py
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ git add demo.py
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ git commit --no-verify -m "inject lint error"
[master cb2a17d] inject lint error
 1 file changed, 4 insertions(+), 3 deletions(-)
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ git push origin HEAD
===== 质量门禁: 格式校验 =====
would reformat demo.py
would reformat calc.py
would reformat test_calc.py

Oh no! 💥💔💥
3 files would be reformatted.
===== 质量门禁: 静态检查 =====
calc.py:4:1: E302 expected 2 blank lines, found 1
calc.py:7:1: E302 expected 2 blank lines, found 1
demo.py:1:12: E201 whitespace after '('
demo.py:1:14: E202 whitespace before ')'
demo.py:2:5: F841 local variable 'unused_var' is assigned to but never used
demo.py:4:1: W391 blank line at end of file
test_calc.py:4:1: E302 expected 2 blank lines, found 1
test_calc.py:7:1: E302 expected 2 blank lines, found 1
test_calc.py:11:1: E302 expected 2 blank lines, found 1
test_calc.py:14:1: E302 expected 2 blank lines, found 1
test_calc.py:17:1: W391 blank line at end of file
===== 质量门禁: 单元测试 =====
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/gry/week3/ex1_precommit
plugins: cov-7.1.0
collected 4 items

test_calc.py .... [100%]

===== 4 passed in 0.02s =====
❌ 质量门禁未通过, 阻断本次推送!
error: 无法推送一些引用到 '/home/gry/week3/ci_remote.git'
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$

```

图 16: 实例 5 (a) 注入错误后门禁拦截推送

```

(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ black calc.py test_calc.py
demo.py
reformatted calc.py
reformatted test_calc.py

All done! ✨ 🍰 ✨
2 files reformatted, 1 file left unchanged.
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ nano test_calc.py
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ flake8 calc.py test_calc.py
demo.py
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ git add calc.py test_calc.py
demo.py
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ git commit -m "fix all format issues"
black.....Passed
flake8.....Passed
[master fa827ee] fix all format issues
3 files changed, 33 insertions(+), 3 deletions(-)
create mode 100644 calc.py
create mode 100644 test_calc.py
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ git push origin HEAD
===== 质量门禁：格式校验 =====
All done! ✨ 🍰 ✨
3 files would be left unchanged.
===== 质量门禁：静态检查 =====
===== 质量门禁：单元测试 =====
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/gry/week3/ex1_precommit
plugins: cov-7.1.0
collected 4 items

test_calc.py .... [100%]

===== 4 passed in 0.01s =====
✅ 三级门禁全部通过，推送成功
枚举对象中：14，完成。
对象计数中：100% (14/14)，完成。
使用 4 个线程进行压缩
压缩对象中：100% (12/12)，完成。
写入对象中：100% (14/14)，1.52 KiB | 1.52 MiB/s，完成。
总共 14 (差异 0)，复用 0 (差异 0)，包复用 0
To /home/gry/week3/ci_remote.git
* [new branch] HEAD -> master
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$

```

图 17: 实例 5 (b) 修复后三级门禁通过

3.6 实例 6：正则查找替换 Markdown 项目符号

题目：练习 regex 查找替换，把讲义中的 - 项目符号替换为 *。直接替换所有 - 是不正确的，因为文件中 - 还有日期、连字符等非项目符号用途；用 ^- 限定行首的列表标记。

```

gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ ls
calc.py code-quality.md demo.py helloworld _pytest_ test_calc.py venv.exe
gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ grep -n '^-' code-quality.md
11:
12:date: 2026-01-23
13:---
14:There are a variety of tools and techniques that support developers in writing high quality code. In this lecture, we'll cover:
15:- [Formatting](#formatting)
16:- [Linting](#linting)
17:- [Testing](#testing)
18:- [Pre-commit hooks](#pre-commit-hooks)
19:- [Continuous Integration](#continuous-integration)
20:- [Command runners](#command-runners)
21:As a bonus topic, we'll also cover [regular expressions](#regular-expressions), a cross-cutting topic that has applications in code quality (e.g., for running a subset of tests that match a pattern) as well as other domains like IDEs (e.g., for search and replace).
22:Many of these tools will be language-specific (e.g., the [ruff](https://docs.astral.sh/ruff/) linter/formatter for Python). In some cases, tools will support multiple languages (e.g., the [Prettier](https://prettier.io/) code formatter). These concepts, however, are near universal — you can find code formatters, linters, testing libraries, and so on for any programming language.
23:Code auto formatters automatically prettify surface syntax. This way, you can focus on the more deep and challenging problems, while the auto-formatting tool handles mundane details such as consistency of `""` versus `'''` syntax for strings, having spaces surrounding binary operators (`x + y` instead of `x+y`), having `import` statements in sorted order, and avoiding over-length lines. One major benefit of code formatters is that they standardize code style across all developers working on a codebase.
24:Some tools such as Prettier are highly configurable(https://prettier.io/docs/configuration); you should check in the configuration file into [version control](/2026/version-control/) for your project. Other tools, such as [Black](https://github.com/psf/black) and [gofmt](https://pkg.go.dev/cmd/gofmt) have limited or no configurability, to reduce [bitsheddin](https://en.wikipedia.org/wiki/Law_of_triviality).
25>You can set up [IDE integration](/2026/development-environment/#code-intelligence-and-language-servers) with your code formatter, so that your code will be auto-formatted as you type or when you save a file. You can also add an [EditorConfig](https://editorconfig.org/) file to your project, which communicates to your IDE certain project level settings like indent size for each file type.
26:Linters come equipped with lists of "rules", with presets that can be configured on a project-level basis. Some linter rules produce false positives, so you can disable them on a per-file or per-line basis.
27:Good linters will have built-in help or documentation that explains each linter rule — what the rule is looking for, why it's bad, and what's a better alternative for the code pattern. For example, see the documentation for the [SIM102](https://docs.astral.sh/ruff/rules/collapsible-if/) rule in [ruff](https://docs.astral.sh/ruff/) which catches unnecessarily nested `if` statements in Python code.
28:Aside from language-specific linters, another tool that might come in handy is [sengrep](https://github.com/sengrep/sengrep), a "semantic grep" tool that works at the AST level (rather than character level, like grep) and supports many languages. You can use sengrep to easily write custom linter rules for your projects. For example, if you wanted to prevent the dangerous `subprocess.Popen(..., shell=True)` in Python, you could find that code pattern with:
29:sengrep -l python -e "subprocess.Popen(..., shell=True, ...)"
30:You can write tests for chunks of code at different levels of granularity: _unit tests_ for individual functions, _integration tests_ for interaction between modules or services, and _functional tests_ for end-to-end scenarios. You can do "test-driven development", where you write tests before you write any implementation code. When you find bugs in your code, you can write _regression tests_, so you'll catch if the functionality ever breaks in the future. You can write _property-based tests_, pioneered in [QuickCheck](https://hackage.haskell.org/package/QuickCheck) in Haskell, and implemented in many libraries, like [Hypothesis](https://hypothesis.readthedocs.io/) for Python. Which approach to testing is right depends on your project; likely, you will adopt some combination.
31:If your program has external dependencies like a database or web API, it may be helpful to _mock_ those dependencies in your tests, rather than have your code interact with third-party dependencies at test time.
32:Code coverage is a metric by which you can measure how good your tests are. Code coverage looks at which lines of your code are executed when your tests are run, so you can ensure you are covering all code paths. Code coverage tools can show you line-by-line coverage to guide you in writing tests. Services such as [Codecov](https://app.codecov.io) provide web interfaces for tracking and viewing code coverage over the history of a project.
33:Like any metric, code coverage is not perfect; don't over-index on coverage, focus on writing high-quality tests.
34:Pre-commit hooks
35:[Git pre-commit hooks](https://git-scm.com/book/en/v2/Customizing-Git-Git-Hooks), made easier by the [pre-commit](https://pre-commit.com/) framework, automatically run user-specified code prior to every git commit. Projects commonly use pre-commit hooks to run formatters and linters, and sometimes tests, automatically before every commit, to ensure that committed code matches the project code style and is free of certain issues.
36:Continuous Integration (CI) services like [GitHub Actions](https://github.com/features/actions) can run scripts for you every time you push code (or on every pull request, or on a schedule). Developers commonly use CI services to run code quality tools including formatters, linters, and tests. For compiled languages, you can ensure code compiles; for statically typed languages, you can make sure it type checks. Running CI every push of new commits can catch errors introduced into the main version of the code; running on pull requests can catch issues with contributor submissions; running on a schedule can catch issues with external dependencies (e.g., a developer accidentally releases a breaking change as [semver-compatible](/2026/shipping-code/releases-versioning/)).
37:Because CI scripts run separately from developer machines, you can easily run long-running jobs there. This can be leveraged, for example, to run a _matrix_ of tests across different operating systems and programming language versions to ensure that the software works properly across all of them.
38:Generally, the script running in CI will not directly make changes to your code: it will run tools in "check-only" mode rather than "fix" mode, so for example, the auto-formatter will raise an error when the code is not compliant with the format.

```

图 18: 实例 6 (a) 替换前 grep 查看多种用途

```

gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ grep -n '2026-01-23' code-quality.md
7:date: 2026-01-23
gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ grep -c '^-' code-quality.md
0
gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$

```

图 19: 实例 6 (b) 替换后验证普通横线未被改动

3.7 实例 7: Makefile 的 clean 构建目标

题目: 在 paper.pdf 的 Makefile 中实现 clean 目标, 用 `rm -f` 清理生成的 PDF 与辅助文件, 并将构建目标声明为 .PHONY; 可用 `git ls-files` 区分源文件与生成物。

```
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ git add Makefile paper.tex
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ git ls-files
Makefile
paper.tex
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ make
pdflatex paper.tex
This is pdfTeX, Version 3.141592653-2.6-1.40.25 (TeX Live 2023/Debian) (preloaded format=pdflatex)
restricted \write18 enabled.
entering extended mode
(./paper.tex
LaTeX2e <2023-11-01> patch level 1
L3 programming layer <2024-01-22>
(/usr/share/texlive/texmf-dist/tex/latex/base/article.cls
Document Class: article 2023/05/17 v1.4n Standard LaTeX document class
(/usr/share/texlive/texmf-dist/tex/latex/base/size10.clo))
(/usr/share/texlive/texmf-dist/tex/latex/l3backend/l3backend-pdfTeX.def)
No file paper.aux.
[1{/var/lib/texmf/fonts/map/pdftex/updmap/pdftex.map}] (./paper.aux) </usr/sha
re/texlive/texmf-dist/fonts/type1/public/amsfonts/cm/cmr10.pfb>
Output written on paper.pdf (1 page, 14132 bytes).
Transcript written on paper.log.
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ ls
Makefile  paper.aux  paper.log  paper.pdf  paper.tex
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$
```

图 20: 实例 7 (a) make 构建生成产物

```
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ make clean
rm -f paper.pdf paper.aux paper.log
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ ls
Makefile  paper.tex
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$
```

图 21: 实例 7 (b) make clean 清理产物

3.8 实例 8: Cargo 依赖版本约束语法

题目: 学习 Rust 构建系统的依赖管理, 对每种语法 (尖号、波浪号、通配符、比较、多个版本要求) 构建一个实际场景。在 Cargo.toml 中为 serde、rand、toml、regex、log 分别配置 ^1.0.188、~0.8.5、0.8.*、>=1.7.0、>=0.4.14, <0.5 并注释场景含义。


```
gry@gry-VMware-Virtual-Platform:~$ mkdir -p ~/week3/cargo_lab
gry@gry-VMware-Virtual-Platform:~$ cd ~/week3/cargo_lab
gry@gry-VMware-Virtual-Platform:~/week3/cargo_lab$ nano Cargo.toml
gry@gry-VMware-Virtual-Platform:~/week3/cargo_lab$ cat Cargo.toml
[package]
name = "version_lab"
version = "0.1.0"
edition = "2021"

[dependencies]
# 1. 尖号 ^ : ^1.2.3 表示 >=1.2.3 且 <2.0.0 (同一主版本内自动用新版本)
# 场景: serde 的 1.x API 稳定, 希望自动获得补丁修复, 又不怕主版本升级破坏代码
serde = "^1.0.188"

# 2. 波浪号 ~ : ~0.8.5 表示 >=0.8.5 且 <0.9.0 (只允许补丁更新)
# 场景: rand 0.8.x 经过验证是稳定的, 不希望 0.9 的大改动影响已有代码
rand = "~0.8.5"

# 3. 通配符 * : 0.8.* 表示 0.8 内的任意版本
# 场景: 只关心主次版本, 具体补丁号无所谓, 有就用最新的
toml = "0.8.*"

# 4. 比较运算符: >=1.7.0 表示至少这个版本
# 场景: regex 从 1.7.0 开始才有需要的某个特性, 版本不能更低
regex = ">=1.7.0"

# 5. 多个版本要求组合: 逗号同时限制上下界
# 场景: 版本不能太低 (缺功能), 也不能太高 (可能有破坏性变更)
log = ">=0.4.14, <0.5"
gry@gry-VMware-Virtual-Platform:~/week3/cargo_lab$
```

图 22: 实例 8: Cargo.toml 五种版本约束及场景

3.9 实例 9: pre-commit 钩子构建检查

题目: 编写 pre-commit 钩子, 在提交前执行 `make paper.pdf`, 构建失败则拒绝提交, 避免产生包含不可构建版本的提交。

结果: 故意把 `\begin{document}` 拼错为 `\begin{doc}`, 提交时 `pdflatex` 报 LaTeX Error, 钩子终止提交; 修复后再次提交成功。

```
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ git config user.name
gry
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ git config user.email
179649847@qq.com
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ nano .git/hooks/pre-commit
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ chmod +x .git/hooks/pre-commit
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ nano paper.tex
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ git add paper.tex
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ git commit -m "test broken build"
==== 提交前构建检查 ====
pdflatex paper.tex
This is pdfTeX, Version 3.141592653-2.6-1.40.25 (TeX Live 2023/Debian) (preloaded format=pdflatex)
 restricted \write18 enabled.
entering extended mode
(./paper.tex
LaTeX2e <2023-11-01> patch level 1
L3 programming layer <2024-01-22>
(/usr/share/texlive/texmf-dist/tex/latex/base/article.cls
Document Class: article 2023/05/17 v1.4n Standard LaTeX document class
(/usr/share/texlive/texmf-dist/tex/latex/base/size10.clo))

! LaTeX Error: Environment doc undefined.

See the LaTeX manual or LaTeX Companion for explanation.
Type H <return> for immediate help.
...

l.2 \begin{doc}
?
! Emergency stop.
...

l.2 \begin{doc}

! ==> Fatal error occurred, no output PDF file produced!
Transcript written on paper.log.
make: *** [Makefile:2: paper.pdf] 错误 1
❌ 构建失败, 拒绝提交!
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$
```

图 23: 实例 9 (a) 构建失败拒绝提交

```
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ nano paper.tex
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ git add paper.tex
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ git commit -m "fix paper"
==== 提交前构建检查 ====
pdflatex paper.tex
This is pdfTeX, Version 3.141592653-2.6-1.40.25 (TeX Live 2023/Debian) (preloaded format=pdflatex)
restricted \write18 enabled.
entering extended mode
(./paper.tex
LaTeX2e <2023-11-01> patch level 1
L3 programming layer <2024-01-22>
(/usr/share/texlive/texmf-dist/tex/latex/base/article.cls
Document Class: article 2023/05/17 v1.4n Standard LaTeX document class
(/usr/share/texlive/texmf-dist/tex/latex/base/size10.clo))
(/usr/share/texlive/texmf-dist/tex/latex/l3backend/l3backend-pdfTeX.def)
No file paper.aux.
[1{/var/lib/texmf/fonts/map/pdftex/updmap/pdftex.map}] (./paper.aux) </usr/sha
re/texlive/texmf-dist/fonts/type1/public/amsfonts/cm/cmr10.pfb>
Output written on paper.pdf (1 page, 14132 bytes).
Transcript written on paper.log.
✅ 构建成功，允许提交
[master (根提交) a108636] fix paper
2 files changed, 11 insertions(+)
create mode 100644 Makefile
create mode 100644 paper.tex
```

图 24: 实例 9 (b) 修复后提交成功

3.10 实例 10: shellcheck 检查 shell 脚本

题目：用 shellcheck 对仓库中所有 shell 文件做静态检查。在 test.sh 中故意写入等号两侧空格、变量未加双引号等错误，shellcheck 报出 SC1007、SC2086 等规则；修复后检查无输出。

```
gry@gry-VMware-Virtual-Platform:~$ mkdir -p ~/week3/shell_lab
gry@gry-VMware-Virtual-Platform:~$ cd ~/week3/shell_lab
gry@gry-VMware-Virtual-Platform:~/week3/shell_lab$ nano test.sh
gry@gry-VMware-Virtual-Platform:~/week3/shell_lab$ shellcheck test.sh

In test.sh line 2:
name= "test"
    ^-- SC1007 (warning): Remove space after = if trying to assign a value (for empty string, use var='' ... ).

In test.sh line 3:
echo $name
    ^---^ SC2154 (warning): name is referenced but not assigned.
    ^---^ SC2086 (info): Double quote to prevent globbing and word splitting.

Did you mean:
echo "$name"

For more information:
https://www.shellcheck.net/wiki/SC1007 -- Remove space after = if trying to...
https://www.shellcheck.net/wiki/SC2154 -- name is referenced but not assign...
https://www.shellcheck.net/wiki/SC2086 -- Double quote to prevent globbing ...
gry@gry-VMware-Virtual-Platform:~/week3/shell_lab$
```

图 25: 实例 10 (a) shellcheck 报错

```
gry@gry-VMware-Virtual-Platform:~/week3/shell_lab$ nano test.sh
gry@gry-VMware-Virtual-Platform:~/week3/shell_lab$ shellcheck test.sh
gry@gry-VMware-Virtual-Platform:~/week3/shell_lab$ shellcheck *.sh
gry@gry-VMware-Virtual-Platform:~/week3/shell_lab$
```

图 26: 实例 10 (b) 修复后批量检查通过

3.11 实例 11: proselint 检查 Markdown 文档

题目: 对仓库中所有.md 文件执行 proselint 写作风格检查。在 test.txt 中写入含 very、utilize 等不规范用词的句子，proselint 报出 weasel_words.very 和 misc.greylint 规则；干净文本检查后无告警。

```
(venv_pro) gry@gry-VMware-Virtual-Platform:~/week3/proselint_lab$ proselint check article.md
(venv_pro) gry@gry-VMware-Virtual-Platform:~/week3/proselint_lab$
```

图 27: 实例 11 (a) proselint 报错

```
(venv_pro) gry@gry-VMware-Virtual-Platform:~/week3/proselint_lab$ nano test.txt
(venv_pro) gry@gry-VMware-Virtual-Platform:~/week3/proselint_lab$ proselint check test.txt
/home/gry/week3/proselint_lab/test.txt:1:7: weasel_words.very: Substitute 'damn' every time you're inclined to write 'very'; your editor will delete it and the writing will be just as it should be.
/home/gry/week3/proselint_lab/test.txt:1:106: misc.greylint: Use of 'utilize'. Do you know anyone who needs to utilize the word utilize?
(venv_pro) gry@gry-VMware-Virtual-Platform:~/week3/proselint_lab$
```

图 28: 实例 11 (b) 干净文本检查通过

4 解题感悟

这一周的内容明显比前面更接近真实工程：格式化、lint、测试覆盖率、持续集成，每一项都是在给代码上“保险”。我的第一个坑还是老问题——环境。装 pre-commit 时 Ubuntu 直接报 externally-managed-environment，全局 pip 装不了，后来才想到用虚拟环境解决；这类问题看报错就能定位，但第一次遇到时还是会懵。

做覆盖率练习时感受最明显：一开始只写了加法和减法的测试，跑出来只有 62%，HTML 报告里 div 函数整段标红，我才直观理解“覆盖率低意味着代码没被跑过”。补上除法的正常和除零两个用例后到 100%，但这个 100% 也提醒我，覆盖率只代表执行到了，不代表测对了边界，真正要补的其实是负数、精度这类情况。

还有几个容易忽略的小坑：proselint 新版本命令格式变成了 proselint check，直接按旧教程敲文件名会报错；Makefile 里命令前面必须是 Tab 而不是空格；git 钩子写好后要 chmod +x 才会执行。这些错误本身都不难，难的是报错信息一出来能不能沉住气一条条读。

这周最大的收获是把前面的零散命令串成了“质量门禁”的思路：格式化器管排版、linter 管规范、测试管功能，再靠 pre-commit 和 CI 在提交和推送时自动把关。做 AI 辅助修

复的练习时我也意识到，工具给的代码只能当草稿，每一步改动都要自己 diff 检查，确认没有破坏原有逻辑才能提交。